

Exercícios — Aula 06 — Busca binária iterativa

Técnicas de Programação - 2026/02

Última atualização: July 31, 2026

Instruções

- Use a convenção deste handout: intervalo fechado, com candidatos de **inicio** até **fim**, inclusive.

Exercício 1 — Simulação com alvo presente

Simule a busca por 42 em:

$v = [3, 8, 12, 19, 25, 31, 42, 57, 68]$

Algorithm 1: BuscaBinariaIterativa

Result: Executa BUSCA-BINARIA-ITERATIVA.

Input : array ordenado v , value alvo

Output: índice do alvo ou -1

```
inicio  $\leftarrow$  0
fim  $\leftarrow$  TAMANHO( $v$ ) - 1
while inicio  $\leq$  fim do
    meio  $\leftarrow$  inicio + INTEIRO((fim - inicio)/2)
    if  $v[\text{meio}] = \text{alvo}$  then
        return meio
    end
    if  $v[\text{meio}] < \text{alvo}$  then
        inicio  $\leftarrow$  meio + 1
    end
    else
        fim  $\leftarrow$  meio - 1
    end
end
return -1
```

passo	inicio	fim	meio	$v[\text{meio}]$
1				
2				
3				

Qual é o retorno?

Exercício 2 — Simulação com alvo ausente

Use o mesmo algoritmo e o mesmo array do exercício anterior. Agora busque 4.

passo	inicio	fim	meio	v[meio]
1				
2				
3				
4				

1. Em qual momento o intervalo fica vazio?
2. Por que o retorno é -1?
3. Quais posições foram descartadas sem comparação direta com o alvo?

Exercício 3 — Calculando o meio

Complete a tabela usando:

$$\text{meio} \leftarrow \text{inicio} + \text{INTEIRO}((\text{fim} - \text{inicio}) / 2)$$

inicio	fim	quantidade de candidatos	meio
0	8		
0	3		
5	8		
2	2		
7	12		
10	11		

Depois, explique por que `inicio <= fim` significa que ainda existe pelo menos um candidato.

Exercício 4 — Quando a busca binária é válida?

Marque os arrays em que a busca binária pode ser usada diretamente para buscar um valor.

- [0, 0, 0, 1, 1, 1]
- [1, 0, 0, 1, 1, 1]
- [3, 8, 12, 19, 25]
- [3, 12, 8, 19, 25]
- []
- [7]
- ["ana", "bia", "carla"], buscando por ordem alfabética

Para cada array marcado como inválido, explique qual pré-condição falhou.

Exercício 5 — Laço infinito

O pseudocódigo abaixo contém um erro comum.

Algorithm 2: BuscaBinariaComErro

Result: Executa BUSCA-BINARIA-COM-ERRO.

Input : array ordenado v , value alvo

Output: índice ou -1

```
inicio  $\leftarrow$  0
fim  $\leftarrow$  TAMANHO(v) - 1
while inicio  $\leq$  fim do
    meio  $\leftarrow$  inicio + INTEIRO((fim - inicio)/2)
    if  $v[\textit{meio}] = \textit{alvo}$  then
        return meio
    end
    if  $v[\textit{meio}] < \textit{alvo}$  then
        inicio  $\leftarrow$  meio
    end
    else
        fim  $\leftarrow$  meio
    end
end
return -1
```

1. Simule para $v = [3, 8]$, alvo = 9.
2. Em qual estado o algoritmo para de avançar?
3. Corrija o código.

Exercício 6 — Escrevendo a versão booleana

Escreva pseudocódigo para CONTEM-BINARIO.

Contrato:

- entrada: array ordenado `v`, valor `alvo`;
- saída: **VERDADEIRO** se o alvo aparece, **FALSO** caso contrário;
- use busca binária iterativa;
- mantenha intervalo fechado.

Teste com:

- `v = [2, 5, 9, 14, 20]`, `alvo = 2`;
- `v = [2, 5, 9, 14, 20]`, `alvo = 20`;
- `v = [2, 5, 9, 14, 20]`, `alvo = 7`;
- `v = []`, `alvo = 2`.

Exercício 7 — Pior caso e comparação com busca linear

Considere arrays ordenados de tamanho n .

1. Para $n = 8$, crie um alvo que force a busca binária a continuar até o intervalo ficar vazio.
2. Quantas comparações com $v[\text{meio}]$ acontecem nesse exemplo?
3. Quantas comparações uma busca linear faria no pior caso com $n = 8$?
4. Repita a comparação para $n = 16$ usando uma estimativa, sem simular tudo.
5. Explique por que a busca binária é descrita como $O(\log n)$.